

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN 1

Ma Trận và Cơ Sở của Tính Toán Khoa Học

Môn học: Toán Ứng Dụng và Thống Kê

Mã môn: MTH00051

Học kỳ: HỌC KỲ 2, 2025 – 2026

Thông tin giảng viên

GV Thực hành: ThS. Võ Nam Thục Đoan, ThS. Lê Nhựt Nam

E-mail: {vntdoan, lnnam}@fit.hcmus.edu.vn

Tài liệu này dành riêng cho mục đích học thuật.

Mục lục

Giới Thiệu Đồ Án	3
1 Phần 1: Phép khử Gauss và Các Ứng Dụng	4
1.1 Cơ Sở Lý Thuyết	4
1.1.1 Phương pháp khử Gauss và Gauss–Jordan	4
1.1.2 Khử Gauss có chọn phần tử chốt (Partial Pivoting)	5
1.1.3 Tính Định Thức qua Khử Gauss	5
1.1.4 Ma Trận Nghịch Đảo bằng Gauss–Jordan	5
1.1.5 Hạng Ma Trận và Cơ Sở	5
1.2 Thuật Toán	5
1.3 Yêu Cầu Cài Đặt Python	6
1.4 Tiêu Chí Đánh Giá — Phần 1	7
2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim	8
2.1 Chéo Hóa Ma Trận (Diagonalizable Matrix)	8
2.2 Các Kỹ Thuật Phân Rã — Chọn Một	8
2.3 Yêu Cầu Trực Quan Hóa với Manim	10
2.4 Tiêu Chí Đánh Giá — Phần 2	10
3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng (Điểm cộng)	11
3.1 Lý Thuyết Sai Số và Tính Ổn Định Số	11
3.1.1 Số Điều Kiện (Condition Number)	11
3.1.2 So Sánh Các Phương Pháp	11
3.1.3 Phương Pháp Lập Gauss–Seidel	11
3.2 Yêu Cầu Cài Đặt và Phân Tích	12
3.3 Tiêu Chí Đánh Giá — Phần 3	12
4 Yêu Cầu Chung và Hướng Dẫn Nộp Bài	13
4.1 Cấu Trúc Báo Cáo	13
4.2 Cấu Trúc Thư Mục Nộp Bài	13
4.3 Yêu Cầu Kỹ Thuật	14
4.4 Phân Công Nhóm và Đạo Đức Học Thuật	14

4.5	Thang Điểm Tổng Hợp	14
	Tài Liệu Tham Khảo	16

Giới Thiệu Đồ Án

Mục tiêu tổng quát

Đồ án này tập trung vào ba nhóm kỹ thuật cốt lõi của đại số tuyến tính số trên máy tính:

1. **Phép khử Gauss** — nền tảng của việc giải hệ phương trình tuyến tính, tính định thức, nghịch đảo và hạng ma trận.
2. **Phân rã ma trận (Matrix Decompositions)** và **Chéo hóa (Diagonalization)** — hiểu bản chất hình học và ứng dụng trực quan hóa.
3. **Ổn định số học và chi phí tính toán** — đánh giá thực nghiệm các phương pháp số trong thực tế.

Sinh viên được kỳ vọng không chỉ *hiểu lý thuyết* mà còn có khả năng *cài đặt từ đầu* (from scratch) bằng Python, *trực quan hóa* bằng Manim, và *phân tích thực nghiệm* về hiệu năng thuật toán.

Các công cụ cho phép sử dụng trong đồ án:

- **Python:** Ngôn ngữ cài đặt chính. Khuyến khích sinh viên sử dụng phiên bản 3.10 hoặc cao hơn.
- **NumPy, SciPy, SymPy:** Kiểm chứng và so sánh kết quả (không dùng để cài đặt thuật toán).
- **math, fractions:** Hỗ trợ xử lý sai số, hiển thị kết quả tính toán (không bắt buộc, không dùng để cài đặt thuật toán).
- **Manim Community:** Trực quan hóa và tạo video demo. Khuyến khích sinh viên sử dụng phiên bản [stable v.20.1](#).
- **Matplotlib:** Vẽ biểu đồ phân tích hiệu năng.
- **Jupyter Notebook:** Trình bày kết quả thực nghiệm.

Lưu ý

Sinh viên **không được** sử dụng trực tiếp các hàm có sẵn như `numpy.linalg.solve`, `numpy.linalg.inv`, `scipy.linalg.qr`, `scipy.linalg.lu`, `sympy.linsolve`, hay các phương thức có sẵn như `echelon_form`, `rref`, v.v... cho phần cài đặt thuật toán. Các thư viện này chỉ được dùng để **kiểm chứng** (verification) kết quả.

1. Phần 1: Phép khử Gauss và Các Ứng Dụng

Tóm tắt yêu cầu Phần 1: Cài đặt phép khử Gauss có partial pivoting từ đầu bằng Python, rồi sử dụng để: **(1)** giải hệ phương trình tuyến tính, **(2)** tính định thức, **(3)** tìm ma trận nghịch đảo, **(4)** tính hạng và tìm cơ sở.

1.1. Cơ Sở Lý Thuyết

1.1.1. Phương pháp khử Gauss và Gauss–Jordan

Xét hệ phương trình tuyến tính $\mathbf{Ax} = \mathbf{b}$ với $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$.

Định nghĩa 1.1 (Ma trận tăng cường). Ma trận tăng cường (augmented matrix) của hệ $\mathbf{Ax} = \mathbf{b}$ là:

$$[\mathbf{A} \mid \mathbf{b}] = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix} \quad (1)$$

Định nghĩa 1.2 (Các phép biến đổi dòng sơ cấp). Ba phép biến đổi dòng sơ cấp (elementary row operations) trên ma trận:

- $R_i \leftrightarrow R_j$: Hoán đổi dòng i và dòng j .
- $R_i \leftarrow c \cdot R_i$, với $c \neq 0$: Nhân dòng i với hằng số c .
- $R_i \leftarrow R_i + c \cdot R_j$: Cộng c lần dòng j vào dòng i .

Ba phép biến đổi này **không làm thay đổi tập nghiệm** của hệ phương trình.

Định nghĩa 1.3 (Dạng bậc thang dòng — REF). Ma trận $\mathbf{U}$ ở dạng bậc thang dòng (Row Echelon Form, viết tắt REF) nếu:

- Mọi dòng toàn số 0 nằm phía dưới các dòng khác 0.
- Phần tử chỉ huy (pivot) của mỗi dòng khác 0 nằm bên phải pivot của dòng ngay trên nó.

Định nghĩa 1.4 (Dạng bậc thang dòng rút gọn — RREF). Ma trận $\mathbf{R}$ ở dạng bậc thang dòng rút gọn (Reduced REF, viết tắt RREF) nếu ngoài điều kiện REF còn có:

- Mỗi pivot bằng 1.
- Mỗi cột chứa pivot có tất cả các phần tử còn lại bằng 0.

1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Trong cài đặt thực tế, cần áp dụng *khử Gauss có chọn phần tử chốt* hay *partial pivoting* để đảm bảo tính ổn định số học.

Định lý 1.1 (Partial Pivoting). *Tại bước khử thứ k , chọn hàng p sao cho:*

$$\left| a_{pk}^{(k)} \right| = \max_{k \leq i \leq n} \left| a_{ik}^{(k)} \right|, \quad (2)$$

rồi hoán đổi dòng k và dòng p trước khi thực hiện khử. Kỹ thuật này giảm thiểu đáng kể sai số làm tròn (round-off error) phát sinh trong tính toán dấu phẩy động.

1.1.3. Tính Định Thức qua Khử Gauss

Sau khi đưa ma trận $\mathbf{A}$ về dạng tam giác trên $\mathbf{U}$ bằng phép khử Gauss với s lần hoán đổi dòng, định thức được tính theo công thức:

$$\det(\mathbf{A}) = (-1)^s \cdot \prod_{i=1}^n u_{ii}. \quad (3)$$

1.1.4. Ma Trận Nghịch Đảo bằng Gauss–Jordan

Định lý 1.2. *Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ khả nghịch khi và chỉ khi $\det(\mathbf{A}) \neq 0$. Nghịch đảo $\mathbf{A}^{-1}$ được tìm bằng cách thực hiện biến đổi dòng đồng thời trên ma trận ghép $[\mathbf{A} \mid \mathbf{I}_n]$ cho đến khi đạt dạng $[\mathbf{I}_n \mid \mathbf{A}^{-1}]$.*

1.1.5. Hạng Ma Trận và Cơ Sở

Định nghĩa 1.5 (Hạng ma trận). *Hạng (rank) của ma trận $\mathbf{A}$, ký hiệu $\text{rank}(\mathbf{A})$, bằng số dòng khác 0 trong dạng REF của $\mathbf{A}$, hoặc tương đương, bằng số cột pivot.*

Từ dạng RREF, ta xác định được ba không gian con quan trọng:

- **Không gian cột** (column space) $\mathcal{C}(\mathbf{A})$: sinh bởi các cột pivot của $\mathbf{A}$ gốc.
- **Không gian dòng** (row space) $\mathcal{R}(\mathbf{A})$: sinh bởi các dòng khác 0 trong RREF.
- **Không gian nghiệm** (null space) $\mathcal{N}(\mathbf{A})$: tập nghiệm của $\mathbf{A}\mathbf{x} = \mathbf{0}$.

1.2. Thuật Toán

Thuật toán 1.1 (Phép khử Gauss với Partial Pivoting). **Đầu vào:** Ma trận $\mathbf{A} \in \mathbb{R}^{n \times m}$, vector $\mathbf{b} \in \mathbb{R}^n$.

Đầu ra: Nghiệm $\mathbf{x}$, số lần hoán đổi s , danh sách chỉ số pivot.

1. Tạo ma trận tăng cường $M = [\mathbf{A} \mid \mathbf{b}]$, đặt $s = 0$.

2. **Với** $k = 1, 2, \dots, n$:

- Tìm $p = \arg \max_{k \leq i \leq n} |M_{ik}|$. Nếu $|M_{pk}| = 0$: báo lỗi không tồn tại pivot tại cột k (hệ không có nghiệm duy nhất); nếu $|M_{pk}| < \varepsilon$: cảnh báo pivot gần bằng 0, hệ có thể không ổn định số học (ill-conditioned).
- Nếu $p \neq k$: hoán đổi dòng $k \leftrightarrow p$, tăng $s \leftarrow s + 1$.
- Với** $i = k + 1, \dots, n$: tính nhân tử $\ell_{ik} = M_{ik}/M_{kk}$, cập nhật $M_{i,\cdot} \leftarrow M_{i,\cdot} - \ell_{ik} \cdot M_{k,\cdot}$.

3. **Thế ngược**: Giải hệ tam giác trên $\mathbf{U}\mathbf{x} = \mathbf{c}$ từ dưới lên:

$$x_i = \frac{1}{M_{ii}} \left(M_{i,n+1} - \sum_{j=i+1}^n M_{ij} x_j \right), \quad i = n, n-1, \dots, 1. \quad (4)$$

Sinh viên cần đưa ra công thức nghiệm tổng quát trong trường hợp vô số nghiệm.

1.3. Yêu Cầu Cài Đặt Python

Yêu cầu cài đặt — Phần 1

Sinh viên cài đặt **từ đầu** (không dùng NumPy/SciPy/SymPy) các hàm sau:

- `gaussian_eliminate(A, b)` — Trả về ma trận sau khi khử, nghiệm $\mathbf{x}$, số lần hoán đổi.
- `back_substitution(U, c)` — Giải hệ tam giác trên.
- `determinant(A)` — Tính $\det(\mathbf{A})$ qua khử Gauss.
- `inverse(A)` — Tính $\mathbf{A}^{-1}$ bằng phương pháp Gauss–Jordan.
- `rank_and_basis(A)` — Tính hạng và cơ sở của các không gian cột, dòng, nghiệm.
- `verify_solution(A, x, b)` — Kiểm chứng kết quả bằng NumPy.

1.4. Tiêu Chí Đánh Giá — Phần 1

Tiêu chí	Mô tả	Điểm
Cài đặt Gauss cơ bản	Chạy đúng, có kiểm thử rõ ràng	1.5
Partial Pivoting	Xử lý đúng khi pivot nhỏ hoặc bằng 0	0.5
Tính định thức	Đúng dấu và giá trị	0.5
Ma trận nghịch đảo	Gauss–Jordan, kiểm thử $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$	1.0
Hạng và cơ sở	Đúng rank, trả về cơ sở các không gian	1.0
Kiểm chứng	So sánh kết quả với NumPy/SciPy	0.5
Tổng Phần 1		5.0

2. Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

Tóm tắt yêu cầu Phần 2: Sinh viên **chọn một** kỹ thuật phân rã ma trận (QR, LU, SVD hoặc Cholesky), cài đặt bằng Python, và tạo **video demo** bằng Manim trực quan hóa quá trình phân rã cùng với chéo hóa ma trận.

2.1. Chéo Hóa Ma Trận (Diagonalizable Matrix)

Định nghĩa 2.1 (Ma trận chéo hóa được). Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ chéo hóa được (diagonalizable) nếu tồn tại ma trận $\mathbf{P}$ khả nghịch và ma trận đường chéo $\mathbf{D}$ sao cho:

$$\mathbf{A} = \mathbf{P} \mathbf{D} \mathbf{P}^{-1}, \quad \mathbf{D} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n), \quad (5)$$

trong đó λ_i là các giá trị riêng (eigenvalues) và các cột của $\mathbf{P}$ là các vector riêng tương ứng.

Định lý 2.1 (Điều kiện chéo hóa). Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ chéo hóa được khi và chỉ khi $\mathbf{A}$ có n vector riêng độc lập tuyến tính. Điều kiện đủ: $\mathbf{A}$ có n giá trị riêng phân biệt.

Ứng dụng quan trọng của chéo hóa trong tính toán khoa học:

$$\mathbf{A}^k = \mathbf{P} \mathbf{D}^k \mathbf{P}^{-1}, \quad \mathbf{D}^k = \text{diag}(\lambda_1^k, \dots, \lambda_n^k). \quad (6)$$

Điều này giúp giảm chi phí tính lũy thừa ma trận từ $O(n^3 \log k)$ xuống $O(n^2)$ sau khi đã có phân tích chéo hoá.

2.2. Các Kỹ Thuật Phân Rã — Chọn Một

Sinh viên tìm hiểu và chọn một trong các kỹ thuật phân rã ma trận được giới thiệu dưới đây:

Tùy chọn A: Phân rã LU (LU Decomposition)

Dạng tổng quát: $\mathbf{PA} = \mathbf{LU}$

- P:** Ma trận hoán vị (permutation matrix)
- L:** Ma trận tam giác dưới với đường chéo bằng 1
- U:** Ma trận tam giác trên

Ý nghĩa: LU chính là phép khử Gauss được viết dưới dạng ma trận. Các nhân tử ℓ_{ik} dùng trong bước khử chính là các phần tử của $\mathbf{L}$.

Ứng dụng: Giải hiệu quả nhiều hệ $\mathbf{Ax}_1 = \mathbf{b}_1, \mathbf{Ax}_2 = \mathbf{b}_2, \dots$ với cùng một ma trận $\mathbf{A}$.

Chi phí tính toán: $O(n^3)$ cho bước phân rã, $O(n^2)$ cho mỗi lần giải hệ.

Tùy chọn B: Phân rã QR (QR Decomposition)**Dạng tổng quát: $A = QR$**

- $A \in \mathbb{R}^{m \times n}$
- $Q \in \mathbb{R}^{m \times n}$: Ma trận trực chuẩn ($Q^T Q = I$)
- $R \in \mathbb{R}^{n \times n}$: Ma trận tam giác trên với các phần tử trên đường chéo chính đều dương, khả nghịch.

Phương pháp cài đặt: Gram–Schmidt cổ điển (CGS) hoặc Householder reflections.**Công thức Gram–Schmidt:**

$$\mathbf{q}_k = \frac{\mathbf{a}_k - \sum_{j=1}^{k-1} (\mathbf{a}_k^T \mathbf{q}_j) \mathbf{q}_j}{\left\| \mathbf{a}_k - \sum_{j=1}^{k-1} (\mathbf{a}_k^T \mathbf{q}_j) \mathbf{q}_j \right\|}. \quad (7)$$

Ứng dụng: Bài toán bình phương tối thiểu (least squares), thuật toán tìm giá trị riêng (QR algorithm).**Tùy chọn C: Phân rã SVD (Singular Value Decomposition)****Dạng tổng quát: $A = U \Sigma V^T$**

- $U \in \mathbb{R}^{m \times m}$: Ma trận trực giao (left singular vectors)
- $\Sigma \in \mathbb{R}^{m \times n}$: Ma trận đường chéo $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$, với σ_i là singular value của A
- $V \in \mathbb{R}^{n \times n}$: Ma trận trực giao (right singular vectors)

Liên hệ với chéo hóa: $\sigma_i = \sqrt{\lambda_i(A^T A)}$, và SVD tương đương chéo hóa của $A^T A$.**Ứng dụng:** Nén dữ liệu, PCA, xấp xỉ hạng thấp:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (\text{xấp xỉ tốt nhất hạng } k). \quad (8)$$

Tùy chọn D: Phân rã Cholesky**Điều kiện áp dụng:** A đối xứng xác định dương (Symmetric Positive Definite — SPD).**Dạng tổng quát:** $A = LL^T$, với L là ma trận tam giác dưới có đường chéo dương.

Công thức tính lặp:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2}, \quad L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} \right), \quad i > j. \quad (9)$$

Chi phí tính toán: Chỉ bằng $\approx \frac{1}{2}$ chi phí LU, đây là ưu điểm lớn khi làm việc với ma trận SPD trong thực tế.

2.3. Yêu Cầu Trực Quan Hóa với Manim

Yêu cầu Manim — Phần 2

Video demo Manim (tối thiểu 2 phút, tối đa 30 phút) phải bao gồm:

- Giới thiệu bài toán:** Hiển thị ma trận **A** cụ thể, nêu rõ bài toán phân rã cần thực hiện.
- Trực quan hóa quá trình phân rã** (tùy theo phương pháp đã chọn):
 - LU:** Minh họa từng bước khử Gauss, nhân tử ℓ_{ij} , hình thành **L** và **U**.
 - QR:** Trực quan Gram–Schmidt trong không gian 2D/3D, biểu diễn vector trực giao hóa.
 - SVD:** Trực quan hóa phép biến đổi hình học rotate–scale–rotate trên hình tròn đơn vị.
 - Cholesky:** Hiển thị tính chất SPD của **A**, từng bước tính **L**.
- Chéo hóa:** Hiển thị giá trị riêng, vector riêng và phép phân tích **A** = **PDP**^{−1}.

2.4. Tiêu Chí Đánh Giá — Phần 2

Tiêu chí	Mô tả	Điểm
Lý thuyết phân rã	Trình bày đầy đủ, chính xác công thức	0.5
Cài đặt Python	Đúng, có kiểm chứng bằng NumPy	1.5
Chéo hóa ma trận	Cài đặt giá trị riêng và chéo hóa	0.5
Video Manim	Hoạt ảnh rõ ràng, đủ nội dung	2.5
Tổng Phần 2		5.0

Nếu sinh viên cài đặt LU (+0.5đ), SVD (+1đ), Cholesky (+1đ).

3. Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng (Điểm cộng)

Tóm tắt yêu cầu Phần 3: So sánh các phương pháp giải $\mathbf{Ax} = \mathbf{b}$, bao gồm Gauss (Phần 1), phân rã (Phần 2) và ít nhất một phương pháp lặp. Phân tích tính ổn định số và chi phí tính toán qua thực nghiệm.

3.1. Lý Thuyết Sai Số và Tính Ổn Định Số

3.1.1. Số Điều Kiện (Condition Number)

Định nghĩa 3.1 (Số điều kiện). Số điều kiện của ma trận $\mathbf{A}$ đối với chuẩn p là:

$$\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \cdot \|\mathbf{A}^{-1}\|_p. \quad (10)$$

Đối với chuẩn spectral (chuẩn 2): $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$.

Định lý 3.1 (Phân tích sai số nghiệm). Nếu $\mathbf{Ax} = \mathbf{b}$ là nghiệm đúng và $\mathbf{Ax} = \mathbf{b} + \delta\mathbf{b}$ là nghiệm tính với nhiễu dữ liệu, thì:

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \cdot \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}. \quad (11)$$

Khi $\kappa(\mathbf{A})$ lớn, hệ bị điều kiện kém (ill-conditioned): sai số nhỏ trong dữ liệu đầu vào gây ra sai số lớn trong nghiệm.

3.1.2. So Sánh Các Phương Pháp

Phương pháp	Loại	Điều kiện áp dụng	Chi phí	Ổn định
Gauss (Partial Pivot)	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
LU với pivot	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
QR (Householder)	Trực tiếp	Mọi $\mathbf{A}$ (kể cả chữ nhật)	$O(n^3)$	Rất cao
Cholesky	Trực tiếp	$\mathbf{A}$ đối xứng xác định dương	$O(n^3/6)$	Rất cao
Gauss–Seidel	Lặp	Chéo trội hàng / SPD	$O(k n^2)$	Trung bình

3.1.3. Phương Pháp Lặp Gauss–Seidel

Phân rã $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ (đường chéo $\mathbf{D}$, tam giác dưới thuần $\mathbf{L}$, tam giác trên thuần $\mathbf{U}$). Ta có công thức lặp:

$$\mathbf{x}^{(k+1)} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D} + \mathbf{L})^{-1}\mathbf{b}. \quad (12)$$

Nếu ta viết theo từng thành phần thì công thức lặp sẽ có dạng như sau:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n. \quad (13)$$

Điều kiện hội tụ: $\mathbf{A}$ chéo trội chặt hàng, tức $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ với mọi i .

3.2. Yêu Cầu Cài Đặt và Phân Tích

Yêu cầu — Phần 3

- Cài đặt** ít nhất 3 phương pháp giải $\mathbf{Ax} = \mathbf{b}$ (bao gồm Gauss từ Phần 1 và phân rã từ Phần 2).
- Thực nghiệm** với ma trận ngẫu nhiên kích thước $n \in \{50, 100, 200, 500, 1000\}$:
 - Đo thời gian thực thi (trung bình 5 lần chạy).
 - Đo sai số tương đối: $\|\mathbf{Ax} - \mathbf{b}\|_2 / \|\mathbf{b}\|_2$.
 - Vẽ đồ thị log-log: thời gian vs n và so sánh với đường lý thuyết $O(n^3)$.
- Phân tích ổn định** với hai loại ma trận:
 - Ma trận Hilbert H_n (số điều kiện rất lớn — ill-conditioned).
 - Ma trận ngẫu nhiên SPD (số điều kiện nhỏ — well-conditioned).
- Báo cáo:** Trình bày trong Jupyter Notebook với bảng số liệu, biểu đồ có chú thích đầy đủ.

3.3. Tiêu Chí Đánh Giá — Phần 3

Tiêu chí	Mô tả	Điểm
Cài đặt phương pháp lặp	Đúng, có kiểm tra điều kiện hội tụ	0.5
Thực nghiệm thời gian	Đủ kích thước, có đồ thị log-log	0.5
Phân tích ổn định số	Ma trận Hilbert vs ma trận ngẫu nhiên	0.5
Nhận xét và kết luận	Phân tích có chiều sâu, có số liệu	0.25
Trình bày Notebook	Rõ ràng, có visualization đầy đủ	0.25
Tổng Phần 3		2.0

4. Yêu Cầu Chung và Hướng Dẫn Nộp Bài

4.1. Cấu Trúc Báo Cáo

Báo cáo viết bằng $\text{\LaTeX}$ hoặc Markdown (xuất ra PDF), bao gồm các mục:

1. **Trang bìa:** Họ và tên, MSSV, nhóm, giảng viên hướng dẫn.
2. **Mục lục.**
3. **Phần 1, 2, 3:** Theo cấu trúc đã nêu ở trên.
4. **Kết luận:** Tóm tắt kết quả đạt được, bài học rút ra, khó khăn gặp phải.
5. **Tài liệu tham khảo:** Ít nhất 5 tài liệu (sách, bài báo hoặc tài liệu chính thống).
6. **Phụ lục** (nếu có): Bảng số liệu chi tiết, code bổ sung.

4.2. Cấu Trúc Thư Mục Nộp Bài

Sinh viên có thể cấu trúc đồ án theo gợi ý như sau:

```
1 Group_<ID>/
2 |-- README.md
3 |-- requirements.txt
4 |-- report/
5 |   |-- report.pdf
6 |   `-- report.tex
7 |-- part1/
8 |   |-- gaussian.py
9 |   |-- determinant.py
10 |   |-- inverse.py
11 |   |-- rank_basis.py
12 |   `-- part1_demo.ipynb
13 |-- part2/
14 |   |-- decomposition.py
15 |   |-- diagonalization.py
16 |   |-- manim_scene.py
17 |   `-- demo_video.mp4
18 `-- part3/
19     |-- solvers.py
20     |-- benchmark.py
21     `-- analysis.ipynb
```

Listing 1: Cấu trúc thư mục nộp bài đồ án

4.3. Yêu Cầu Kỹ Thuật

- Sử dụng ngôn ngữ lập trình Python, viết code rõ ràng dễ hiểu (clean code), có chú thích và giải thích cho việc cài đặt thuật toán.
- Sử dụng Manim để trực quan hóa và minh họa các bước thuật toán thông qua video.
- Sinh viên sử dụng `requirements.txt` để quản lý dependencies (nếu sử dụng pip) hoặc `*.yaml` (nếu sử dụng conda).
- Video: Định dạng MP4, độ phân giải phù hợp $\geq 720p$, thời lượng ≥ 2 phút.
- Kiểm thử/ đánh giá: Mỗi hàm có ít nhất 5 test cases, bao gồm các trường hợp đặc biệt (edge cases).

4.4. Phân Công Nhóm và Đạo Đức Học Thuật

Lưu ý

- Báo cáo phải ghi rõ **phân công công việc** của từng thành viên trong nhóm.
- Giảng viên sẽ chọn lựa một số nhóm để vấn đáp nếu cần thiết.
- **Nghiêm cấm** sao chép code hoặc báo cáo từ nhóm khác hay từ Internet mà không trích dẫn nguồn.
- Sử dụng công cụ AI (ChatGPT, GitHub Copilot, v.v.) để *gợi ý* là được phép, nhưng sinh viên phải **hiểu và giải thích được toàn bộ code** mình nộp.
- Vi phạm đạo đức học thuật dẫn đến **điểm 0 toàn bộ đồ án**.

4.5. Thang Điểm Tổng Hợp

Phần	Nội dung	Điểm tối đa	Trọng số
1	Phép khử Gauss và ứng dụng	5.0	38.5%
2	Phân rã ma trận + Video Manim	6.0	46.2%
3	Phân tích hiệu năng và ổn định	2.0	15.4%
Tổng cộng		13.0	100%

Điểm cuối cùng sẽ được quy về thang điểm 10.

Gọi P1, P2, P3 lần lượt là điểm phần 1, 2 và 3 (trong đó Phần 3 là điểm cộng thêm).

$$\text{Điểm cuối cùng} = \min\left(\frac{P1 + P2}{11} \times 10 + P3, 10\right).$$

(14)

Tóm tắt sản phẩm nộp bài

- ☐ **Báo cáo** `report.pdf` (bắt buộc)
- ☐ **Source code** đầy đủ kèm `README.md` và `requirements.txt`
- ☐ **Video Manim** `demo_video.mp4` (Phần 2, bắt buộc)
- ☐ **Jupyter Notebooks**: `part1_demo.ipynb` và `analysis.ipynb`
- ☐ **Nộp qua**: Moodle của Khoa
- ☐ **Hạn nộp**: **[ngày 20/4/2025, trước 23:59]**

Tài Liệu Tham Khảo

- [1] Gilbert Strang. *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.
- [2] Lloyd N. Trefethen & David Bau III. *Numerical Linear Algebra*. SIAM, 1997.
- [3] Gene H. Golub & Charles F. Van Loan. *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.
- [4] Cleve Moler. *Numerical Computing with MATLAB*. SIAM, 2004. Truy cập miễn phí: <https://www.mathworks.com/moler/chapters.html>
- [5] Manim Community Developers. *Manim — Mathematical Animation Framework*, v0.18. 2024. <https://docs.manim.community>
- [6] Jake VanderPlas. *Python Data Science Handbook*. O'Reilly, 2016. <https://jakevdp.github.io/PythonDataScienceHandbook/>
- [7] 3Blue1Brown. *Essence of Linear Algebra* (chuỗi video). YouTube, 2016. https://www.youtube.com/playlist?list=PLZHQ0b0WTQDPD3MizzM2xVFItgF8hE_ab